Конспект: подготовка данных и оценка моделей

Nov 3, 2026 · @Эльнур Гасанов

1. Factorize и One-Hot Encoding (OHE)

Модели работают с числами, а категориальные признаки («red», «green») — это строки. Оба способа превращают категории в числа, но по-разному.

pd.factorize() присваивает каждой уникальной категории целое число (код) в порядке первого появления: первая встреченная категория получает 0, следующая новая — 1 и так далее. Функция возвращает два объекта: массив кодов и список уникальных значений. Получается один столбец с числами.

One-Hot Encoding создаёт отдельный столбец для каждой категории. В нужном столбце стоит 1, в остальных 0. Получается столько столбцов, сколько категорий (в pandas — `pd.get_dummies()`, в sklearn — `OneHotEncoder`).

Пример: признак `color` = red, green, blue, green, red.

```
codes, uniques = pd.factorize(df["color"])
# codes   -> [0, 1, 2, 1, 0]
# uniques -> ['red', 'green', 'blue']

pd.get_dummies(df["color"])   # столбцы в алфавитном порядке
```

color	factorize	OHE: blue	OHE: green	OHE: red
red	0	0	0	1
green	1	0	1	0
blue	2	1	0	0
green	1	0	1	0
red	0	0	0	1

Главное различие. Factorize вводит порядок: для модели 2 больше 1, а 1 больше 0, хотя «blue» не «больше» «green». Для линейных моделей и k-NN это ложная закономерность. Поэтому:

- Factorize подходит, когда у категорий есть естественный порядок (низкий / средний / высокий) или для деревьев решений, которым порядок кодов не мешает. Также им кодируют целевую переменную (метки классов).
- OHE подходит для категорий без порядка (цвет, город), особенно для линейных моделей и нейросетей. Минус: при множестве категорий (сотни городов) число столбцов сильно растёт.

2. SimpleImputer

Многие модели sklearn не принимают NaN (пропуски) и падают с ошибкой. `SimpleImputer` заменяет пропуски одним значением, посчитанным по столбцу. Как и масштабирование, он имеет `fit` (запоминает значение по обучающим данным) и `transform` (подставляет его).

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median")
X_train = imp.fit_transform(X_train)
X_test = imp.transform(X_test)
```

Четыре стратегии:

- **mean** — среднее арифметическое столбца. Только для чисел. Чувствительно к выбросам.
- **median** — медиана (середина отсортированных значений). Только для чисел. Устойчива к выбросам.
- **most_frequent** — самое частое значение (мода). Работает и для чисел, и для строк.
- **constant** — любое заданное число или строка из `fill_value` (например, 0 или "unknown"). Полезно, когда сам факт пропуска несёт смысл.

Пример: возраст = 20, 25, NaN, 35, 100 (известны четыре значения).

Стратегия	Расчёт	Чем заполнится NaN
mean	(20 + 25 + 35 + 100) / 4	45
median	значения 20, 25, 35, 100; середина = (25 + 35) / 2	30
constant, fill_value=0	задано вручную	0

Выброс 100 сильно тянет среднее вверх (45), а медиана почти не меняется (30): поэтому при выбросах чаще берут median.

Для категорий: город = Баку, Гянджа, Баку, NaN, Сумгаит. Стратегия most_frequent заполнит пропуск значением «Баку» (оно встречается чаще всех). Стратегия constant с `fill_value="unknown"` поставит «unknown».

3. MinMaxScaler и StandardScaler

Зачем масштабировать. Признаки имеют разные единицы и диапазоны: возраст 20–60, зарплата 500–5000. Модели, основанные на расстояниях (k-NN, SVM, кластеризация) и на градиентном спуске (линейная регрессия, нейросети), будут считать признак с большими числами более важным, хотя это просто масштаб. Масштабирование приводит признаки к сопоставимым величинам. Деревьям решений оно обычно не нужно.

MinMaxScaler сжимает значения в диапазон [0, 1]: минимум становится 0, максимум 1.

```
x' = \frac{x - x_{min}}{x_{max} - x_{min}}
```

StandardScaler (стандартизация) сдвигает среднее в 0 и делает стандартное отклонение равным 1.

```
z = \frac{x - \mu}{\sigma}
```

Здесь μ — среднее признака, σ — его стандартное отклонение. Диапазон не ограничен: z показывает, на сколько σ значение отстоит от среднего.

Пример: x = 10, 20, 30, 40, 50. Здесь min = 10, max = 50, μ = 30, σ = √200 ≈ 14,14 (сумма квадратов отклонений 1000, делим на 5).

x	MinMax	Standard
10	0	−1,414
20	0,25	−0,707
30	0,5	0
40	0,75	0,707
50	1	1,414

Разница. MinMax гарантирует границы [0, 1], но один выброс сдвигает min или max и «сплющивает» остальные значения. StandardScaler менее зависит от одной крайней точки и лучше подходит для моделей, ожидающих данные около нуля, но не ограничивает диапазон. MinMax удобен, когда нужен жёсткий диапазон (например, для нейросетей и изображений).

Почему fit только на train. `fit` вычисляет параметры (min, max, μ, σ). Если посчитать их по всем данным, в них попадёт информация из тестовой выборки: это утечка данных (data leakage), и оценка качества окажется завышенной. В реальности новых данных мы не видим заранее. Правильно:

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)  # параметры считаем по train
X_test_s = scaler.transform(X_test)        # к test только применяем
```

То же правило относится к SimpleImputer и другим преобразованиям с `fit`.

4. Метрики регрессии: MAE, MSE, RMSE, R^2

Обозначения: y_i — истинное значение, $\hat{y}_i$ — предсказание модели, $\bar{y}$ — среднее истинных значений, n — число объектов. Ошибка одного объекта — разница $y_i - \hat{y}_i$.

MAE (Mean Absolute Error, средняя абсолютная ошибка). Берём модуль каждой ошибки и усредняем. Показывает, на сколько единиц в среднем ошибается модель, в тех же единицах, что и целевая переменная.

```
MAE = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_i\right|
```

MSE (Mean Squared Error, средний квадрат ошибки). Ошибки возводятся в квадрат и усредняются. Единицы — квадрат исходных (рубли², метры²), поэтому смысл сложнее интерпретировать.

```
MSE = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2
```

RMSE (Root Mean Squared Error). Корень из MSE возвращает метрику в исходные единицы.

```
RMSE = \sqrt{MSE}
```

R^2 (коэффициент детерминации). Показывает, какую долю разброса целевой переменной объясняет модель, по сравнению с наивной моделью, которая всегда предсказывает среднее $\bar{y}$.

```
R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}
```

Какие значения лучшие. Для MAE, MSE и RMSE лучше меньше: идеал равен 0 (ошибок нет). Для R^2 лучше больше: идеал равен 1 (модель объясняет всё). $R^2 = 0$ означает «не лучше, чем предсказывать среднее».

Пример. y_true = 3, 5, 2, 7; предсказания y_pred = 2, 5, 4, 10.

Объект	y	ŷ	Ошибка (y − ŷ)	|Ошибка|	Ошибка²
1	3	2	1	1	1
2	5	5	0	0	0
3	2	4	−2	2	4
4	7	10	−3	3	9
Сумма				6	14

- MAE = 6 / 4 = 1,5
- MSE = 14 / 4 = 3,5
- RMSE = √3,5 ≈ 1,87
- R^2: среднее y равно 17 / 4 = 4,25; сумма $(y - \bar{y})^2$ = 1,5625 + 0,5625 + 5,0625 + 7,5625 = 14,75; $R^2 = 1 - 14 / 14{,}75 \approx 0{,}05$. Модель лишь чуть лучше предсказания средним.

Реакция на большие ошибки. MSE и RMSE сильнее реагируют на крупные ошибки, потому что квадрат их увеличивает: ошибка 3 даёт вклад 9, а ошибка 1 только 1. В примере ошибка 3 даёт 9 из 14 в сумме квадратов (64%), но лишь 3 из 6 в сумме модулей (50%). MAE растёт пропорционально ошибке и устойчивее к выбросам. Если большие промахи особенно опасны, смотрят на RMSE; если важна типичная ошибка, на MAE.

Может ли R^2 быть отрицательным? Да. Знаменатель — ошибка наивной модели «всегда ȳ». Если наша модель ошибается сильнее неё (числитель больше знаменателя), дробь больше 1 и $R^2 < 0$. Это означает, что модель хуже простого среднего (например, из-за сильного переобучения или плохих признаков), особенно когда R^2 считают на тестовых данных.

5. Cross-Validation (кросс-валидация)

Что это и зачем. Кросс-валидация — способ оценить качество модели, при котором данные несколько раз делятся на обучающую и проверочную части, а итоговая оценка получается усреднением результатов. Она нужна, потому что одна оценка на одном случайном разбиении зависит от везения: тест мог

случайно оказаться «лёгким» или «трудным». Усреднение по нескольким разбиениям даёт более надёжную оценку, а разброс между ними показывает, насколько результат стабилен.

Как работает K-Fold. Данные делятся на K равных частей (фолдов). Затем K раз повторяется: один фолд служит проверочным, остальные K − 1 — обучающими. В итоге каждый объект ровно один раз побывал в проверке. Результат — среднее K оценок.

Пример: cv=5 и 100 объектов, в каждом фолде по 20.

Итерация	Проверка (test)	Обучение (train)
1	объекты 1–20	21–100 (80 шт.)
2	объекты 21–40	1–20 и 41–100
3	объекты 41–60	1–40 и 61–100
4	объекты 61–80	1–60 и 81–100
5	объекты 81–100	1–80

Пусть R^2 на пяти итерациях получился 0,80; 0,76; 0,84; 0,78; 0,82. Итог: (0,80 + 0,76 + 0,84 + 0,78 + 0,82) / 5 = 4,00 / 5 = 0,80. Модель обучается пять раз, перед каждым обучением преобразования (scaler, imputer) тоже заново делают `fit` только на train-части фолда.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring="r2")
print(scores, scores.mean())
```

Отличие от одного Train/Test. При одном разбиении модель обучается и проверяется один раз: оценка быстрая, но зависит от конкретного разбиения, а часть данных (тест) никогда не участвует в обучении. Кросс-валидация использует данные эффективнее (каждый объект бывает и в обучении, и в проверке), даёт среднее и разброс оценок, но требует примерно в K раз больше времени. Один Train/Test уместен для очень больших данных или как финальная проверка, а кросс-валидация — для подбора модели и параметров на небольших и средних данных.